

Reviewer Pack — Minimal Representation Length of Quantum Groups and Circuit ...

Entry 0bc26292660b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 15:22:39 UTC

Minimal Representation Length of Quantum Groups and Circuit Monotone Width

Entry ID: 0bc26292660b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 15:22:39 UTC

1. Conjecture

Field A (mathematical branch): Quantum Group Theory **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every Boolean circuit C with n inputs, the minimal representation length of its associated quantum group G_C required to express the monomial basis elements is linearly related to its circuit monotone width $w(C)$, such that $|R(G_C)| = \Theta(w(C))$

Rationale (proposer's reasoning):

Quantum groups have been used in algebraic combinatorics and quantum information theory but are rarely applied to complexity theory. Their representation length captures the structure of the group and may reveal hidden complexities within circuits.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4777f91b0c4a3947

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean representation length of quantum groups for circuits with a range of monotone widths is within a factor of two of the monotone width, across at least ten seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - quantum group theory AND boolean circuit monotone width - circuit monotone width IN quantum group theory literature - representation length of quantum groups related to boolean circuit complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define a simple Boolean circuit with varying monotone width
    def generate_circuit(n):
        if n == 1:
            return ['A']
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return [f'OR({left[0]}, {right[0]})', f'AND({left[0]}, {right[0]})']

    # Compute the associated quantum group using its minimal representation
    def compute_quantum_group(circuit):
        if len(circuit) == 1:
            return [circuit[0]]
        else:
            left = compute_quantum_group([circuit[0]])
            right = compute_quantum_group([circuit[2]])
            return left + right

    # Measure the representation length of the quantum group
    def measure_representation_length(group):
        return len(group)

    # Correlate the measured representation length with the circuit's monotone width
    def calculate_monotone_width(circuit):
        if len(circuit) == 1:
            return 1
        else:
            left = calculate_monotone_width([circuit[0]])
            right = calculate_monotone_width([circuit[2]])
            return max(left, right) + 1
```

```

# Generate a set of Boolean circuits with varying monotone width
n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_circuit(n)
    quantum_group = compute_quantum_group(circuit)
    representation_length = measure_representation_length(quantum_group)
    monotone_width = calculate_monotone_width(circuit)

    results.append({
        "n": n,
        "representation_length": representation_length,
        "monotone_width": monotone_width
    })

# Compute the mean and standard deviation of the representation length
total_representation_length = sum(result["representation_length"] for result in results)
mean_representation_length = total_representation_length / len(results)

variance = sum((result["representation_length"] - mean_representation_length) ** 2 for result in results)
std_deviation = math.sqrt(variance / len(results))

# Check if the conjecture is supported
support_fraction = sum(1 for result in results if abs(result["representation_length"] - mean_representation_length) < std_deviation)

return {
    "metric_name": "Representation Length",
    "metric_value": mean_representation_length,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"mean={mean_representation_length}, std={std_deviation}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7e61218.py", line 92, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7e61218.py", line 58, in run_trial
    quantum_group = compute_quantum_group(circuit)
                    ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7e61218.py", line 36, in compute_quantum_group
    right = compute_quantum_group([circuit[2]])
                    ~~~~~^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:
 skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:
 The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12970
2	preregistration	ollama_remote	glm4:latest	0	0	9182
3	novelty	ollama_remote	glm4:latest	0	0	8309
4	novelty	ollama_remote	glm4:latest	0	0	8793
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12783
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8864
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9435
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9741
9	judge	ollama_remote	glm4:latest	0	0	13572

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93651 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/0bc26292660b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0bc26292660b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0bc26292660b.tar.gz (if generated)